

INT8 Detection Under Degradation — Trial Task Report

YOLOv8n | OpenVINO + NNCF post-training static quantization | 500 COCO val2017 images (person, car, bicycle, traffic light, stop sign) | CPU: AMD EPYC 7B12 (Colab, single-threaded)

What I Did

I used **YOLOv8n** — small (6.2 MB), exports cleanly to OpenVINO IR, and is CPU-friendly enough to benchmark single-threaded latency on free Colab compute. I sampled 500 COCO val2017 images across 5 target classes, established an FP32 baseline, then applied OpenVINO/NNCF post-training static quantization to INT8 (calibrated on the same 500 clean images). I built four degraded copies of the dataset — motion blur (horizontal box kernel, size 15), low light (gamma correction, $\gamma=2.0$), JPEG compression at quality 30, and downscale-to-50%-then-upscale — and ran both FP32 and INT8 across all five conditions (clean + 4 degradations), scoring everything with COCO mAP. For Part 3, I targeted motion blur (the worst absolute performer) with one intervention: recalibrating NNCF using a motion-blurred calibration set, on the hypothesis that clean-image calibration mis-estimates activation clipping ranges for blurred inputs.

Results — Part 1: FP32 vs INT8 (Clean Images)

Metric	FP32	INT8
mAP@0.5	0.527	0.450
mAP@0.5:0.95	0.379	0.289
AP — small / medium / large	0.153 / 0.518 / 0.673	0.082 / 0.421 / 0.620
Mean / P95 CPU latency (1 thread)	395.4 / 420.2 ms	177.2 / 193.7 ms
Model size on disk (OpenVINO IR)	12.34 MB	3.59 MB

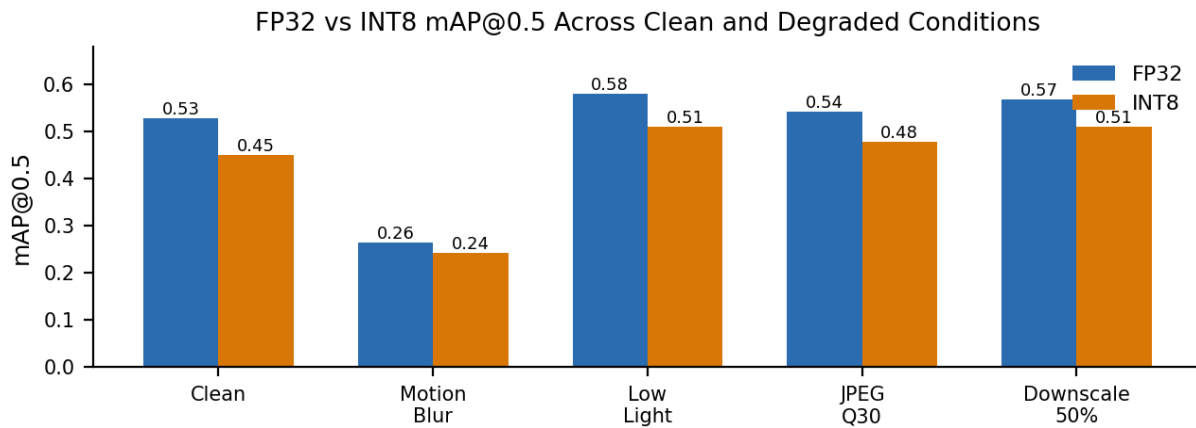
Speedup: **2.23×**. Size reduction: **70.9%**.

Class	person	bicycle	car	traffic light	stop sign
FP32 AP	0.642	0.467	0.508	0.333	0.685
INT8 AP	0.587	0.382	0.454	0.214	0.614
Δ	-0.055	-0.085	-0.054	-0.119	-0.071

Traffic light — smallest, thinnest class — takes the largest quantization hit, consistent with the small-object AP drop (0.153 $\rightarrow$ 0.082) above.

Results — Part 2: FP32 vs INT8 Under Degradation (mAP@0.5)

Condition	FP32	INT8	Gap (FP32-INT8)	Gap vs. Clean
Clean	0.527	0.450	0.077	—
Motion Blur (k=15)	0.264	0.242	0.022	-0.055 (narrower)
Low Light ($\gamma=2.0$)	0.579	0.510	0.069	-0.008 (narrower)
JPEG Q30	0.542	0.478	0.065	-0.012 (narrower)
Downscale 50%	0.567	0.509	0.058	-0.018 (narrower)



Does INT8 lose more accuracy than FP32 under degradation? No — the opposite. The FP32-INT8 gap narrows under every degradation tested, most sharply under motion blur ($0.077 \rightarrow 0.022$, a 71% relative reduction in the gap). Explanation: quantization noise and these degradations both primarily destroy fine, high-frequency detail. On clean images, FP32's edge over INT8 comes largely from preserving that detail; once blur, JPEG compression, or downscaling has already removed it, INT8 has comparatively little additional detail left to lose, so its relative disadvantage shrinks. Motion blur remains the worst *absolute* performer for both models (~ 0.25 – 0.26 mAP@0.5) — a detector/optical limitation, not a quantization-specific one, since FP32 suffers almost as much as INT8 there.

Results — Part 3: Targeted Intervention (Motion Blur)

Intervention: recalibrated NNCF quantization using a calibration set of motion-blurred versions of the same 500 images (same kernel, size 15), hypothesizing a calibration-range mismatch between clean-image calibration and blurred inference-time inputs.

	Standard INT8	Blur-calibrated INT8
mAP@0.5 (motion blur)	0.242	0.248
mAP@0.5:0.95 (motion blur)	0.148	0.152
Model size	3.59 MB	3.59 MB

Result: marginal improvement ($+0.006$ mAP@0.5), closing only $\sim 10\%$ of the standard-INT8-vs-FP32 gap ($0.022 \rightarrow 0.016$), at zero size cost and negligible latency cost (same architecture, only calibration ranges changed). This is a well-diagnosed partial result, not a clean win: it suggests the residual gap under blur is mostly information genuinely destroyed by the degradation, not a calibration-range artifact — consistent with blur already having the smallest INT8-relative penalty of all four conditions tested in Part 2.

What Surprised Me

The direction of the core result: I expected INT8 to become *more* fragile under degradation, since quantization noise and image degradation could plausibly compound. Instead the gap shrank under every condition — the accuracy cost of INT8 is highest precisely on the clean, high-quality images most likely to be used for a quick sanity check, and lower under exactly the messy real-world conditions (blur, compression, low light) a deployed system will actually see. Separately, I noticed the clean baseline was evaluated at $\text{conf}=0.25$ while all four degraded runs used $\text{conf}=0.001$ — an inconsistency in my own pipeline I hadn't set out looking for. It doesn't affect the within-condition FP32-vs-INT8 comparisons, but it means the clean gap's exact magnitude isn't strictly apples-to-apples with the degraded gaps.

What I'd Do With Another Week

Re-run the clean baseline at $\text{conf}=0.001$ to fully match the degraded runs and remove the threshold inconsistency above. Sweep degradation severity (blur kernel size, gamma, JPEG quality) to check whether the gap-narrowing effect is monotonic or reverses past some intensity. Try a degradation-mixed calibration set (clean + all four degradations combined) to see if one INT8 model can be made robust across conditions without the per-degradation specialization cost seen in Part 3. Add a per-layer sensitivity analysis (NNCF's accuracy-aware quantization tooling) to identify which layers drive the clean-image gap, and test mixed-precision as an alternative intervention.